

UNIVERSITY OF BIRMINGHAM

Final Year Project Proposal

Automated AI-Powered Code Review for Students

Yousef Al Dali

November 2025

Abstract

This project is about building an Automated AI-powered code review tool that helps students learn how to code better and faster. The tool will read the student's code, point out errors, and explain why they have done wrong. It will also give a better version of the code and a short summary of what the code does. The idea is to make students understand and learn from their mistakes instead of just fixing them. The project will start with Python and Java and later, if there is time, include a live coaching feature that gives hints while the student is writing code.

---

1. Introduction

1.1 Problem Statement

When students write code, they often make mistakes or use inefficient methods. Traditional code reviews are slow and depend on teachers or TAs. Static tools like linters can find syntax or style issues, but they cannot explain the problem or show how to fix it in a way that helps students learn. This project will build a smart AI system that reviews code like a human tutor, it finds issues, explains them, and teaches the student how to improve.

1.2 Project Objectives

- Create a tool that reads the code and finds bugs, performance issues, and security problems.
- Explain each issue in a simple way and show a better version of the code.
- Give a short summary of what the code does, to help students understand their work.

- Build a web dashboard where students can upload or paste their code and see the results clearly.
- Use CodeBERT to help the system understand code and find similar correct examples.
- Add a live coaching mode later if there is enough time.

1.3 Scope and Modes

The tool will focus on **Python and Java** for the main version. It will have two modes:

1. **Full Review Mode:** When a student uploads code, it gives a full review with explanations, summaries, and code fixes.
2. **Real-Time Coaching (Stretch Goal):** Gives short tips as students write or paste code, like a quick tutor.

1.4 Project Significance

This project can make learning to code faster and easier. It will help students get instant, useful feedback instead of waiting for someone to review their code. It will also help teachers by reducing their workload and keeping feedback consistent. The main goal is not just to fix code but to help students understand *why* something is wrong and how to do it better.

---

2. Literature Review and Related Work

Many researchers have worked on code understanding and automated reviews. Pretrained code models like CodeBERT learn from both natural language and code, which makes them useful for finding patterns or mistakes. AI code review tools like CodeRabbit and Amazon CodeGuru exist, but they are built for professional developers, not students. They usually give fixes but not educational explanations. This project will combine static tools (like Flake8, Bandit, Checkstyle, PMD, and SpotBugs) with AI reasoning to give feedback that teaches.

Research gap: Most tools either only find syntax issues or just suggest code fixes without explaining them. None of them focus on *learning* or *student understanding*. This project aims to fill that gap.

---

3. Methodology

3.1 How the system will work

1. A student uploads or pastes their code into the dashboard. (Note: try VScode extension)
2. The backend runs static analysis using Python and Java linters.
3. CodeBERT helps classify the type of issue and find similar correct examples.
4. The system creates feedback for each issue: what's wrong, why, and how to fix it.
5. It shows the results with explanations, patch previews, and a short summary of what the code does.

3.2 Tools and Technologies

- **Languages:** Python, Java
- **AI Model:** CodeBERT (for code understanding)
- **Linters:** Flake8, Bandit, Checkstyle, PMD, SpotBugs
- **Frontend:** React or Next.js for the dashboard
- **Backend:** FastAPI (Python)
- **Database:** SQLite or MongoDB (for saving results and tests)
- **Version Control:** GitLab (for CI/CD integration and project hosting)

3.3 Features

- Detects bugs, security, and performance issues.
- Gives detailed explanations and code examples.
- Creates a short automatic summary of what the code does.
- Lets students view all comments and patches visually.
- (Later) Real-time hints while typing or pasting code.

---

4. Project Specification

Functional Requirements

1. Upload or paste code for review.
2. Detect bugs, performance, and security issues.
3. Explain each issue in natural language.
4. Suggest better code with patch previews.
5. Provide a short summary of the code.
6. Show all results in a clean dashboard view.
7. Save and export the feedback in JSON format.

Non-Functional Requirements

- The review should finish quickly (under 60 seconds for small code files).
- The explanations must be clear and easy to read.
- The system must not send private code outside the local environment.
- It should be stable and easy to use.

---

5. Evaluation and Validation

The system will be tested with anonymised student code examples. I will measure:

- **Accuracy:** How many detected issues are real and useful.
- **Helpfulness:** If students find the explanations easy to understand.

- **Performance:** How fast it runs.
- **Trust:** If students are confident using the tool.

If time allows, I will also compare the AI tool with a normal linter to see how much better it performs in giving meaningful feedback.

---

6. Implementation Timeline

Weeks 1–3: I will start by understanding the project more and collecting requirements. During this stage I will build the main structure of the system and make the basic connection between the frontend and backend. I will also set up the linters for Python and Java (Flake8, Bandit, Checkstyle, PMD, SpotBugs) and make the detectors and patches for the main rules.

Weeks 4–6: I will work on the first version of the rule pack for Python and Java. Then I will connect everything together and build the first version of the dashboard where students can upload or paste their code and see the results. I will also add the GitLab CI option if I have time.

Weeks 7–9: I will add CodeBERT to the project so it can classify and find similar code examples to help give better feedback. I will also start making a small dataset of anonymised student code and collect the first evaluation results to see how accurate and helpful the tool is.

Weeks 10–12: I will focus on improving the safety of the system so it does not make insecure suggestions or wrong fixes. I will also test the performance and speed of the system and start preparing for evaluation.

Weeks 13–15: I will test the full system with different examples, check how students interact with it, and make small improvements to the interface and feedback explanations. I will also start writing the final report.

Weeks 16–17: I will finish the report and documentation and, if I have time left, I will add the real-time coaching feature to the web editor so it gives quick hints while students are typing or pasting their code.

---

7. Expected Outcomes and Deliverables

By the end of the project, I should have:

- A working AI code review tool for Python and Java.

- A web dashboard that shows the results clearly.
- A small dataset of student code and an evaluation report.
- The final project report and presentation for the inspection.

---

8. Risks and Challenges

- The AI model might give wrong or confusing explanations.
- Time might not be enough to finish everything, especially the live coaching part.
- Running large models locally could be slow.
- Keeping the tool safe and not leaking any student code.

---

9. Summary

This project will create an Automated AI-powered code review tool that acts like a learning assistant for students. It will explain problems, show how to fix them, and help students write better code faster. Starting with Python and Java, it will combine static analysis and CodeBERT to give meaningful feedback that helps students understand, not just correct, their mistakes.